

SSOO PRÁCTICA 3

GRUPO FORMADO POR
JAIME SÁNCHEZ SÁNCHEZ,
CRISTIAN DULUGEAC Y
PAULA DÍAZ COLLADO

ÍNDICE:

INTRODUCCIÓN	2
Principales objetivos:	3
EXPLICACIÓN DE CÓDIGO	4
process_manager.h	4
process_manager.c	4
factory_manager.c	5
queue.h	5
queue.c	6
Makefile	7
Batería de pruebas	8
PRINCIPALES PROBLEMAS	8

INTRODUCCIÓN

En la práctica se plantea la implementación de un sistema jerárquico de control de producción, donde intervienen distintos tipos de hilos. Por un lado, el factory_manager representa el hilo principal del sistema, encargado de leer un fichero de configuración y coordinar la creación de varios process_manager. Cada uno de estos se encarga de gestionar una cinta de transporte virtual en la que se desarrolla una interacción clásica de tipo productor-consumidor. Los elementos se insertan y extraen de una cola circular, simulando el flujo de productos en una línea de montaje.

Esta práctica permite aplicar de manera práctica conceptos teóricos fundamentales del área de sistemas operativos, al tiempo que se desarrollan habilidades clave como el trabajo con hilos POSIX, la programación modular, la depuración de aplicaciones concurrentes y la integración de componentes en un proyecto estructurado.

Principales objetivos:

- Comprender y aplicar los conceptos de programación concurrente mediante hilos POSIX.
- Implementar mecanismos de sincronización como mutexes, variables de condición y semáforos POSIX.
- Diseñar e implementar un sistema productor-consumidor en el que múltiples entidades operan simultáneamente sobre recursos compartidos.
- Gestionar correctamente el uso de estructuras de datos compartidas (colas circulares), evitando condiciones de carrera y errores de concurrencia.
- Familiarizarse con técnicas de depuración y trazado de programas concurrentes.

El conjunto de ficheros presentados conforma una simulación multihilo de una fábrica en la que distintos gestores de procesos (`process_managers`) coordinan la producción y consumo de elementos en cintas transportadoras virtuales (`belts`). Cada cinta tiene un tamaño limitado y se produce una cantidad definida de productos, los cuales son gestionados por hilos productores y consumidores. Para sincronizar y coordinar la ejecución de todos los componentes, se emplean semáforos POSIX y variables de condición. La arquitectura sigue un enfoque jerárquico, en el cual un `factory_manager` (gestor principal) lanza múltiples `process_manager`, que a su vez generan hilos internos de producción y consumo.

EXPLICACIÓN DE CÓDIGO

process_manager.h

Este archivo contiene la definición de la estructura de parámetros que se utiliza para pasar información a los hilos `process_manager`. La estructura parámetros incluye el ID de la cinta, el tamaño de la misma, el número de productos que se deben producir, el índice del proceso y punteros a los semáforos utilizados para la sincronización entre hilos.

Además, se declara la función `process_manager`, la cual es implementada en el archivo `.c` correspondiente y representa el hilo principal que maneja una línea de producción (una cinta transportadora).

process_manager.c

Este archivo implementa la lógica de cada gestor de proceso (`process_manager`), así como los hilos internos de producción (`producer`) y consumo (`consumer`). Su funcionamiento incluye:

- Inicialización de la cinta (cola) y estructuras de sincronización.
- Creación de los hilos `producer` y `consumer`, que trabajan coordinadamente usando un mutex y variables de condición para llenar y vaciar la cinta.
- Uso de semáforos para esperar y notificar al `factory_manager` sobre el estado de cada proceso.
- El productor espera a que la cinta esté vacía antes de producir un lote de productos. Una vez que la cinta está llena, notifica al consumidor.
- El consumidor consume todos los productos del lote, detectando si uno es el último mediante un `flag last`.
- Al finalizar, los hilos son finalizados con `pthread_join` y se limpian los recursos.

factory_manager.c

Este es el archivo principal que contiene la función main y actúa como orquestador del sistema. Su rol incluye:

- Leer un archivo de entrada con la configuración de las cintas y el número de productos.
- Validar la información del archivo de entrada.
- Inicializar estructuras dinámicas para los parámetros, hilos y semáforos.
- Crear un hilo process_manager por cada cinta transportadora configurada.
- Sincronizar el inicio de los process_manager mediante semáforos para garantizar que todos estén listos antes de iniciar la producción.
- Esperar a que cada process_manager finalice su tarea y luego unirse a los hilos.
- Finalmente, liberar todos los recursos utilizados.

queue.h

Este archivo contiene las definiciones relacionadas con la implementación de una cola compartida (belt), donde los elementos producidos se insertan y luego se consumen. Se define una estructura element que representa un producto, incluyendo su número de edición, el identificador de la cinta a la que pertenece y si es el último elemento producido.

También se declaran funciones para manipular la cola, como queue_init, queue_put, queue_get, queue_empty, queue_full y queue_destroy. Estas funciones permiten gestionar la cola de manera segura en un entorno multihilo.

queue.c

Este módulo implementa la cola circular que actúa como cinta transportadora entre los hilos productores y consumidores. Su principal objetivo es ofrecer un mecanismo de almacenamiento intermedio que soporte operaciones seguras de inserción y extracción bajo un modelo concurrente.

Funciones clave implementadas:

- `int queue_init(int size)`: Reserva memoria para la cola e inicializa sus variables de control. Retorna -1 en caso de error.
- `int queue_put(struct element* x)`: Inserta un nuevo elemento en la posición final de la cola. Si está llena, devuelve -1.
- `struct element* queue_get(void)`: Extrae el siguiente elemento del principio de la cola. Si está vacía, devuelve NULL.
- `int queue_empty(void)` y `int queue_full(void)`: Consultan el estado de la cola.
- `int queue_destroy(void)`: Libera los recursos dinámicos utilizados y reinicia las variables globales.

Cabe señalar que este módulo gestiona únicamente el almacenamiento circular, mientras que la protección frente a concurrencia (mutex y variables condicionales) se maneja desde los hilos productor/consumidor en `process_manager.c`.

Batería de pruebas

Nº	Objetivo	Entrada de prueba	Resultado esperado	Resultado obtenido
1	Verificar el correcto funcionamiento	Fichero: 1 10 1 5→ 1 cinta, tamaño 10, 5 elementos.	- Se crea un solo process_manager.- Producir inserta elementos 0...4 y consumer los extrae. Mensajes OK.	Conforme: se generan y consumen 5 elementos en orden, y todos los mensajes OK
2	Manejo de múltiples cintas	Fichero: 3 1 3 2 2 2 3 3 1→ 3 cintas, tamaños y producciones diferentes	- Se crea cada process_manager en orden del fichero. Cada pareja productor-consumidor trabaja concurrente sin interferencias.	Conforme: las trazas de cada cinta están intercaladas pero respetan orden interno y capacidad
3	Caso límite: buffer tamaño 1 Sin bloqueo	Fichero: 2 7 1 3 8 1 2→ cinta 7 tamaño=1, produce 3; cinta 8 tamaño=1, produce 2	- Con buffer=1, producir se bloquea hasta consumo de cada elemento.- No hay pérdidas ni interbloqueos. Mensajes OK	Se observa alternancia put/get en cada cinta; sin bloqueos.
4	Error en fichero:	Fichero: 2 1 5 1 2 3 1→ max=2, pero vienen 3 grupos	Detectar inconsistencia en la lectura.- Mensaje de error: [ERROR][factory_manager] Invalid file.- Retorno -1.	En stderr aparece mensaje de error y el programa termina con código -1.
5	Error de argumentos en process_manager	Fichero: 5 abc 10	Detectar que no se usan números Mensaje de error: [ERROR][process_manager] Arguments not valid.- Retorno -1.	Se imprime en stderr y sale con -1.

PRINCIPALES PROBLEMAS

Los principales problemas que hemos encontrado a la hora de realizar la práctica han sido regular los órdenes de ejecución de cada thread para que sigan el orden puesto por el input, tanto en la ejecución de los process managers y en la queue. Y que esto se vea reflejado en los prints que crean los hilos en su correspondiente orden, que es lo que más ha costado, para ello hemos tenido que cambiar cosas fundamentales de nuestro código como semáforos externos para optar por semáforos en structs pasandolos a todos los procces como atributos. Haciendo que desde el factory se vayan desbloqueando o bloqueando. Gestionando así con bucles de maneras eficiente los prints.

```
Test input7.txt: is running...
The files have the same lines.
Test: input7.txt: PASSED
-----
##### Summary / Resumen #####
Passed tests / Pruebas superadas: 74/74
a0522207@guernika:~/Escritorio/ssoop3tester$
```